

고객을 세그먼테이션하자 [프로젝트] - 이정숙

11-2. 데이터 불러오기

데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
SELECT *  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`  
Limit 10;
```

Job information Results Visualization JSON Execution details Execution graph									
Row	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom	
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom	
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom	
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom	
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom	
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom	
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom	
10	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	2010-12-01 08:34:00 UTC	1.69	13047	United Kingdom	

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT COUNT(*) AS total_rows  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`  
Limit 10;
```

Query results									
Job information Results Visualization JSON Execution details Execution graph									
Row	total_rows								
1	541909								

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
SELECT
COUNT(InvoiceNo) AS InvoiceNo_countnt,
COUNT(StockCode) AS StockCode_count,
COUNT(Description) AS Description_count,
COUNT(Quantity) AS Quantity_countnt,
COUNT(InvoiceDate) AS InvoiceDate_count,
COUNT(UnitPrice) AS UnitPrice_count,
COUNT(CustomerID) AS CustomerID_countnt,
COUNT(Country) AS Country_count,

FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`;
```

Job information		Results	Visualization	JSON	Execution details	Execution graph		
Row	InvoiceNo_countnt	StockCode_count	Description_count	Quantity_countnt	InvoiceDate_count	UnitPrice_count	CustomerID_countnt	Country_count
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```
SELECT
    'Description' AS column_name,
    ROUND(SUM(CASE WHEN Description IS NULL THEN 1 ELSE
0 END) / COUNT(*) * 100, 2) AS missing_percentage
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`
```

UNION ALL

SELECT

```
'CustomerID' AS column_name,  
ROUND(SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0  
END) / COUNT(*) * 100, 2) AS missing_percentage  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da  
ta`;
```

Row	column_name ▼	missing_percenta...
1	Description	0.27
2	CustomerID	24.93

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```
SELECT DISTINCT Description  
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da  
ta`  
WHERE StockCode = '85123A';
```

Row	Description ▼
1	WHITE HANGING HEART T-LIGHT HOLDER
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIGHT HOLDER

결측치 처리

- DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시

```
DELETE FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]];
```

Job information

Results

Execution details

Execution graph

i This statement removed 135,080 rows from data.

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

```
SELECT COUNT(*) AS duplicate_rows
FROM (
    SELECT
        InvoiceNo,
        StockCode,
        Description,
        Quantity,
        InvoiceDate,
        UnitPrice,
        CustomerID,
        Country,
        COUNT(*) AS cnt
    FROM `project-3e5836df-194c-42fe-b66.modulabs_project.
data`
    GROUP BY
        InvoiceNo,
        StockCode,
        Description,
```

```

Quantity,
InvoiceDate,
UnitPrice,
CustomerID,
Country
HAVING COUNT(*) > 1
);

```

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	duplicate_rows ▾				
1	4837				

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기
 - CREATE OR REPLACE TABLE** 구문을 활용하여 모든 컬럼(*)을 **DISTINCT** 한 데이터로 업데이트

```

CREATE OR REPLACE TABLE `project-3e5836df-194c-42fe-b66.
modulabs_project.data` AS
SELECT DISTINCT *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`;

```

Job information	Results	Execution details	Execution graph
 This statement replaced the table named data.			

```

72
73 SELECT COUNT(*) AS duplicate_rows
74 FROM (
75     SELECT
76         InvoiceNo,
77         StockCode,
78         Description,
79         Quantity,
80         InvoiceDate,
81         UnitPrice,
82         CustomerID,
83         Country,
84         COUNT(*) AS cnt
85     FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
86     GROUP BY
87         InvoiceNo,
88         StockCode,
89         Description,
90         Quantity,
91         InvoiceDate,
92         UnitPrice,
93         CustomerID,
94         Country
95     HAVING COUNT(*) > 1
96 );
97

```

✓ Query completed

Using on-demand processing quota

Query results

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	duplicate_rows				
1	0				

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo 의 개수를 출력하기

```

SELECT COUNT(DISTINCT InvoiceNo) AS Count_InvoiceNo
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`;

```

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Count_InvoiceNo				
1	22190				

- 고유한 **InvoiceNo** 를 앞에서부터 100개를 출력하기

```
SELECT DISTINCT InvoiceNo
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
LIMIT 100;
```

Row	InvoiceNo	
1	541431	
2	C541433	
3	537626	
4	542237	
5	549222	
6	556201	
7	562032	
8	573511	
9	581180	
10	539318	
11	541998	
12	548955	
13	568172	
14	577609	
15	543037	
16	544156	
17	545323	
18	545332	

Results per page: 50 ▼ 1 – 50 of 100 |< < > >

- InvoiceNo** 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT DISTINCT *
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

Row	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	C541433	23166	MEDIUM CERAMIC TOP STORA...	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom
2	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	183.75	12352	Norway
3	C545329	M	Manual	-1	2011-03-01 15:47:00 UTC	280.05	12352	Norway
4	C545330	M	Manual	-1	2011-03-01 15:49:00 UTC	376.5	12352	Norway
5	C547388	22701	PINK DOG BOWL	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
6	C547388	22784	LANTERN CREAM GAZEBO	-3	2011-03-22 16:07:00 UTC	4.95	12352	Norway
7	C547388	21914	BLUE HARMONICA IN BOX	-12	2011-03-22 16:07:00 UTC	1.25	12352	Norway
8	C547388	37448	CERAMIC CAKE DESIGN SPOTT...	-12	2011-03-22 16:07:00 UTC	1.49	12352	Norway
9	C547388	84050	PINK HEART SHAPE EGG FRYN...	-12	2011-03-22 16:07:00 UTC	1.65	12352	Norway
10	C547388	22413	METAL SIGN TAKE IT OR LEAVE...	-6	2011-03-22 16:07:00 UTC	2.95	12352	Norway
11	C547388	22645	CERAMIC HEART FAIRY CAKE M...	-12	2011-03-22 16:07:00 UTC	1.45	12352	Norway

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT ROUND(SUM(CASE WHEN Quantity < 0 THEN 1 ELSE 0 EN
D)/COUNT(*) * 100, 1) AS Canceled_Ratio
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`
```

```
-- 선호 버전:
-- SELECT ROUND(COUNTIF(Quantity <0)/COUNT(*) * 100,1) A
S Canceled_Ratio
-- FROM `project-3e5836df-194c-42fe-b66.modulabs_projec
t.data`
```

Job information	Results	Visualization	JSON	Execution details	Execution graph
Row	Canceled_Ratio				
1	2.2				

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기


```
SELECT COUNT(DISTINCT StockCode)
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`;
```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	f0_ ▼					
1		3684				

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기
 - 상위 10개의 제품들을 출력하기

```
SELECT StockCode, COUNT(*) AS sell_cnt
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.da
ta`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```

Job information		Results	Visualization	JS
Row	StockCode	sell_cnt		
1	85123A	2065		
2	22423	1894		
3	85099B	1659		
4	47566	1409		
5	84879	1405		
6	20725	1346		
7	22720	1224		
8	POST	1196		
9	22197	1110		
10	23203	1108		

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```

SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
    r'[0-9]', '')) AS number_count
  FROM `project-3e5836df-194c-42fe-b66.modulabs_project.
  data`
)
WHERE number_count = 0 or number_count = 1;

```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	StockCode	number_count				
1	POST	0				
2	M	0				
3	C2	1				
4	D	0				
5	BANK CHARGES	0				
6	PADS	0				
7	DOT	0				
8	CRUK	0				

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```

SELECT
  ROUND(
    SUM(CASE
      WHEN LENGTH(StockCode) - LENGTH(REGEXP_REPLACE
        (StockCode, r'[0-9]', '')) <= 1
      THEN 1 ELSE 0
    END) / COUNT(*) * 100,
    2
  ) AS percentage
FROM `project-3e5836df-194c-42fe-b66.modulabs_project.data`;

```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	percentage					
1	0.48					

- 제품과 관련되지 않은 거래 기록을 제거하기

```
DELETE FROM project_name.modulabs_project.data
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    # [[YOUR QUERY]]
  );
```

[결과 이미지를 넣어주세요]

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT Description, COUNT(*) AS description_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

- 서비스 관련 정보를 포함하는 행들을 제거하기

```
DELETE
FROM project_name.modulabs_project.data
WHERE
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.data AS
SELECT
  * EXCEPT (Description),
  # [[YOUR QUERY]] AS Description
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

UnitPrice 살펴보기

- UnitPrice의 최솟값, 최댓값, 평균을 구하기

```
SELECT # [[YOUR QUERY]] AS min_price, # [[YOUR QUERY]] AS max_price, # [[YOUR QUERY]] AS avg_price
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기

```
SELECT # [[YOUR QUERY]] AS cnt_quantity, # [[YOUR QUERY]] AS min_quantity, # [[YOUR QUERY]] AS max_quantity, # [[YOUR QUERY]] AS avg_quantity
FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- UnitPrice = 0를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.data AS
SELECT *
FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

11-7. RFM 스코어

Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```
SELECT # [[YOUR QUERY]] AS InvoiceDay, *
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
SELECT
    # [[YOUR QUERY]] AS most_recent_date,
    # [[YOUR QUERY]] AS InvoiceDay,
    *
FROM project_name.modulabs_project.data;
```

[결과 이미지를 넣어주세요]

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```
SELECT
    CustomerID,
    # [[YOUR QUERY]] AS InvoiceDay
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- 가장 최근 일자(**most_recent_date**)와 유저별 마지막 구매일(**InvoiceDay**)간의 차이를 계산하기

```
SELECT
    CustomerID,
    EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay)
AS recency
FROM (
    SELECT
        CustomerID,
        MAX(DATE(InvoiceDate)) AS InvoiceDay
    FROM project_name.modulabs_project.data
    GROUP BY CustomerID
);
```

[결과 이미지를 넣어주세요]

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 **user_r** 이라는 이름의 테이블로 저장하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.us
er_r AS
# [[YOUR QUERY]]
```

[결과 이미지를 넣어주세요]

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```
SELECT
  CustomerID,
  # [[YOUR QUERY]] AS purchase_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
SELECT
  CustomerID,
  # [[YOUR QUERY]] AS item_cnt
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.us
er_rf AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  # [[YOUR QUERY]]
),
```



```
-- (2) 구매한 아이템 총 수량 계산
item_cnt AS (
  # [[YOUR QUERY]]
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN project_name.modulabs_project.user_r AS ur
  ON pc.CustomerID = ur.CustomerID;
```

[결과 이미지를 넣어주세요]

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```
SELECT
  CustomerID,
  # [[YOUR QUERY]] AS user_total
FROM project_name.modulabs_project.data
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

- 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인 (LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.us
er_rfm AS
SELECT
    rf.CustomerID AS CustomerID,
    rf.purchase_cnt,
    rf.item_cnt,
    rf.recency,
    ut.user_total,
    # [[YOUR QUERY]] AS user_average
FROM project_name.modulabs_project.user_rf rf
LEFT JOIN (
    -- 고객 별 총 지출액
    SELECT
        # [[YOUR QUERY]]
    ) ut
ON rf.CustomerID = ut.CustomerID;
```

[결과 이미지를 넣어주세요]

RFM 통합 테이블 출력하기

- 최종 `user_rfm` 테이블을 출력하기

```
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) `user_rfm` 테이블과 결과를 합치기
- 3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH unique_products AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT StockCode) AS unique_products
    FROM project_name.modulabs_project.data
    GROUP BY CustomerID
)
SELECT ur.*, up.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_rfm AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

[결과 이미지를 넣어주세요]

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_data AS
WITH purchase_intervals AS (
    -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
    SELECT
        CustomerID,
        CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE
        ROUND(AVG(interval_), 2) END AS average_interval
    FROM project_name.modulabs_project.data
    GROUP BY CustomerID
)
```

```

FROM (
  -- (1) 구매와 구매 사이에 소요된 일수
  SELECT
    CustomerID,
    DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITI
ON BY CustomerID ORDER BY InvoiceDate), DAY) AS interval_
  FROM
    project_name.modulabs_project.data
  WHERE CustomerID IS NOT NULL
)
GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;

```

[결과 이미지를 넣어주세요]

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기
 - 1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
 - 2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
 - 취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기
(취소 비율은 소수점 두번째 자리)

```

CREATE OR REPLACE TABLE project_name.modulabs_project.user_
data AS

```

```

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    # [[YOUR QUERY]] AS total_transactions,

```

```

        # [[YOUR QUERY]] AS cancel_frequency
    FROM project_name.modulabs_project.data
    # [[YOUR QUERY]]
)

SELECT u.*, t.* EXCEPT(CustomerID), # [[YOUR QUERY]] AS cancel_rate
FROM `project_name.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON # [[YOUR QUERY]];

```

[결과 이미지를 넣어주세요]

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기

```
# [[YOUR QUERY]];
```

[결과 이미지를 넣어주세요]

회고

[회고 내용을 작성해주세요]

Keep :

Problem :

Try :

Row	Description ▼	
1	WHITE HANGING HEART T-LIGHT HOLDER	
2	?	
3	wrongly marked carton 22804	
4	CREAM HANGING HEART T-LIGHT HOLDER	